LAB 12

Example 1:

Intial state: stack is empty

Step 1 ss.enter scope()
global scope created
Output: Enter new scope (depth = 1)

Step 2: ss.declare ("x", INT)
insert x into scope 1 scope 1{x: INT}
Output Declare x in scope 1.

Step 3 ss.declare ('y', float)
insert y into scop1, scope1 { y : FLOAT};
Output: Declare y into scope depth = 1

Step 4. ss.enterscope(); New scope (inner)
Output : Entered New scop (depth = 2

Step 5 ss.declare ("x", FLOAT);
Declare new x in scope 2.
Output: Declared x in scope2 (depth = 2)

Step 6: ss.lookup ("x")
check scope2 → 'x' found.
Output Found 'x' in scope depth 2.

Step 7: s.s.lookup ('y')
not found in scope 2
scope1 → found. Output: Found y in scope 2
return → Float

Step 8: ss.exitscope() → Remove scope2
Output: Exited scope (depth = 1)

Step 9: ss.lookup ("x") scope 1 → found, Output: Found x in scope depth 1

Step 10: ss.lookup ("z") → z in scope 1 → not found
Output "z" not declared (error) Returned type → unknown

Example 2:

creating AST nodes

ASTNode * two = new ASTNode(2);

node created: tree:

value = 2 2

ASTnode * three = new ASTNode(3);

Node create: tree:

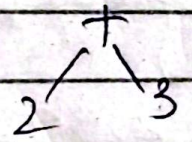
NUMNODE 3

value = 3

ASTNode * plus = new ASTNode ('+', two, three);

Binary operator node created

Tree:



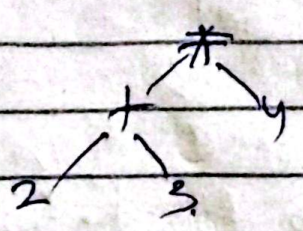
ASTNode * four = new ASTnode(4);

node created Tree:

NUM_NODE 4

value = 4;

ASTNode * mult = newASTNode ('*', plus, four);



Step2: Print AST (printAST(mult))

traversal recursive.

-> print "("
-> left subtree -> (2+3)
-> print (*)'
-> print right subtree -> 4.
Out

Step 3. Evaluate
Evaluation happening
-> 2+3 = 5 -> left subtre
-> 4. -> right subtr
-> 5 * 4 = 20 root.

Final output
AST: ((2+3)*4)
Evaluated value =
20.

~~Explae~~ Example 3.

Step1

AST Expression : a=(b+c)*d

Step2. AST node creation

ASTNode* b =new ASTnode("b");

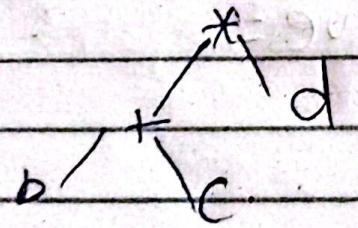
ASTNode* b =newASTNode("c");

" " =new ASTNode('+', b, c);

" " " ('d');

" " " ('*' plus, d);

Final AST structure:



Step3. tac.assign ('a', mult) call inside assign();

ths reg = generate(mult)

now generate recursively traveses the AST

Step 4. Traversin AST

-> Process * node, generat (mult)

First left subtree

-> Process + node, generat (plus)

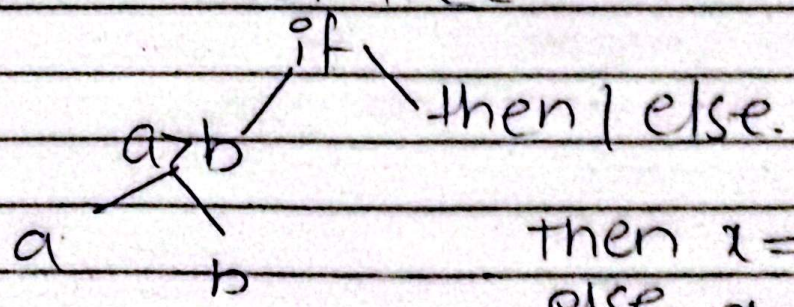
= process ~~leftsubtree~~ left child

-> visit b , generate (b) return -> b
-> visit c , generate(c) - return -> c
-> t1 = b+c temporary , return t1
-> visit d , generat (d) return -> d

-> t2 = t1 * d temporary return t2
-> step 5:
Assign() assignment instructi
a = t2

Example 4:

AST tree



then x = 1
else x = 0

Step1:
genExpr (cond)
condition node -> a > b
temp var -> t1
t1 = a>b -> instruction gene
Step 2: newlabel() -> creates labels
step 3: if t1 == 0 goto L1
if cond is false -> jump to else
step 4 x = 1 -> then block
step5 goto L1 -> skip else Block
step6 L1: -> else Block
step7 x = 0 -> else block
step8 L2: -> end label